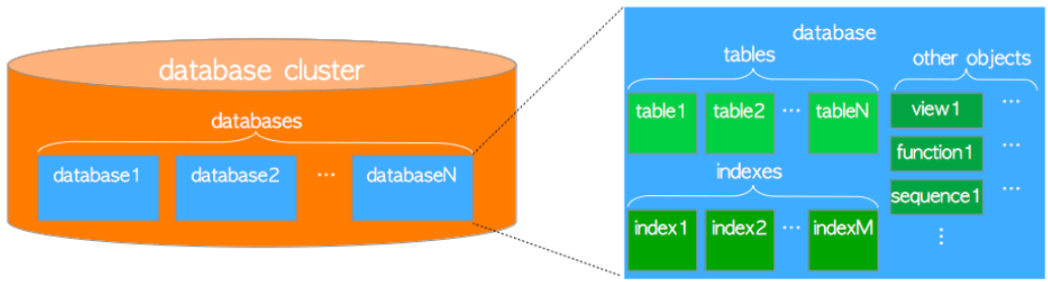


数据库簇、数据库和数据表

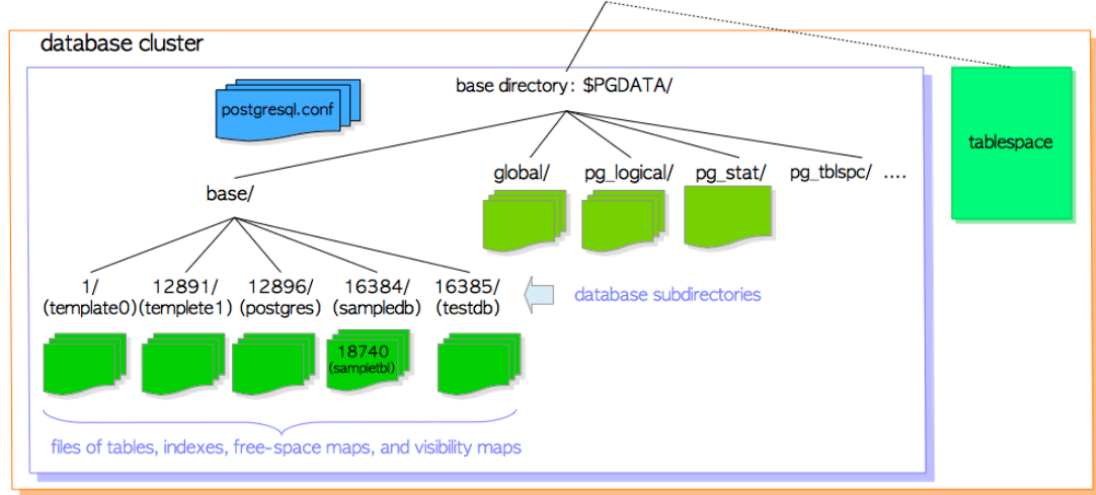
数据库簇的逻辑结构

数据库簇（database cluster）是一组数据库（database）的集合，由一个 PostgreSQL 服务器管理。数据库簇与高可用数据库集群不同，并非意味着“一组数据库服务器”，一个 PostgreSQL 服务器只会在单机上运行并管理单个数据库簇。



数据库簇的物理结构

数据库簇本质上就是一个文件目录，即基础目录。执行 `initdb` 会在指定目录下创建基础目录，从而初始化一个新的数据库簇。PostgreSQL 中的表空间对应一个包含基础目录之外的数据的目录。



数据库簇的布局

主要的文件如下：

file	description
PG_VERSION	包含 PostgreSQL 主版本号
pg_hba.conf	控制 PostgreSQL 客户端认证

file	description
pg_ident.conf	控制 PostgreSQL 用户名映射
postgresql.conf	配置参数
postgresql.auto.conf	存储使用 ALTER SYSTEM 修改的配置参数 (version 9.4 or later)
postmaster.opts	记录服务器上次启动的命令行选项，例如： /home/postgres/pgsql/bin/postgres "-D" "/pgdata/postgres" "-c" "config_file=/pgdata/postgres/postgresql.conf" "-p" "1035"

主要的目录如下:

subdirectories	description
base/	每个数据库对应的子目录都存储在此
global/	数据库集簇范畴的表 (例如 pg_database) 及 pg_control
pg_commit_ts/	事务提交的时间戳数据 (Version 9.5 or later)
pg_clog/	事务提交状态数据 (Version 9.6 or earlier)。在版本10.0中被重命名为 pg_xact
pg_dynshmem/	动态共享内存子系统中使用的文件 (Version 9.4 or later)
pg_logical/	逻辑解码的状态数据 (Version 9.4 or later)
pg_multixact/	多事务状态数据
pg_notify/	LISTEN/NOTIFY 状态数据
pg_replslot/	复制槽数据 (Version 9.4 or later)
pg_serial/	已提交的可串行化事务相关信息 (version 9.1 or later)
pg_snapshots/	导出快照 (version 9.2 or later)。PostgreSQL 函数 pg_export_snapshot 在此子目录中创建的快照信息文件
pg_stat/	统计子系统的永久文件
pg_stat_tmp/	统计子系统的临时文件
pg_subtrans/	子事务状态数据
pg_tblspc/	指向表空间的符号链接
pg_twophase/	两阶段事务的状态文件
pg_wal/	WAL 段文件 (Version 10 or later)，从 pg_xlog 重命名而来
pg_xact/	事务提交状态数据 段文件 (Version 10 or later)，从 pg_clog 重命名而来
pg_xlog/	WAL 段文件 (Version 9.6 or earlier)，它在版本10.0中被重命名为 pg_wal

数据库布局

一个数据库与base子目录下的一个子目录对应

```
select datname, oid from pg_database order by oid;
```

datname	oid
template1	1
template0	13213
postgres	13214

(3 rows)

```
[postgres@localhost base]$ ll
total 36
drwx-----. 2 postgres postgres 8192 Feb 21 18:35 1
drwx-----. 2 postgres postgres 8192 Feb 21 18:35 13213
drwx-----. 2 postgres postgres 8192 Mar 13 21:21 13214
```

表和索引相关文件的布局

每个小于1GB的表或索引都在相应的数据库目录中存储为单个文件。这些数据文件由变量relfilenode管理。

```
select relname, oid, relfilenode from pg_class where relname='tt01';
```

relname	oid	relfilenode
tt01	16386	16386

(1 row)

```
[postgres@localhost base]$ find . -name 16386
./13214/16386
```

relfilenode通常和oid一致。不过，relfilenode会被一些命令（例如TRUNCATE、REINDEX、CLUSTER、VACUUM FULL）所改变。

```
truncate tt01;
```

```
select relname, oid, relfilenode from pg_class where relname='tt01';
```

relname	oid	relfilenode
tt01	16386	16390

(1 row)

```
find . -name 16390
./13214/16390
```

当表和索引的文件大小超过1GB时，PostgreSQL会创建并使用一个名为relfilenode.1的新文件，以此类推，会有relfilenode.2。

```
insert into tt01(id)
select generate_series(1,100000000);

INSERT 0 100000000
```

```
select pg_size_pretty(pg_table_size('tt01'));

pg_size_pretty
-----
3458 MB
(1 row)
```

```
find . -name '16390*'
./13214/16390
./13214/16390_fsm
./13214/16390.1
./13214/16390.2
./13214/16390.3
```

每个表都有两个与之关联的文件，_fsm和_vm分别存储了表文件每个页面上的空闲空间信息与可见性信息。索引没有可见性映射文件，只有空闲空间映射文件。

```
create table tt02(id int );

select pg_relation_filepath('tt02');
pg_relation_filepath
-----
base/13214/16392
(1 row)
```

```
find . -name '16392*'
./13214/16392
```

```
insert into tt02(id) select generate_series(1,100000);
```

```
find . -name '16392*'
./13214/16392
./13214/16392_fsm
```

```
update tt02 set id = id + 1;  
checkpoint;
```

```
find . -name '16392*'  
./13214/16392  
./13214/16392_fsm  
./13214/16392_vm
```

fsm -- free space maps

在第一次对表进行vacuum 时才会建立fsm文件,另外需要提示的是FSM 文件使用的是树形结构来记录, 空闲的页块, 通过代码来看也是从左到右的查找

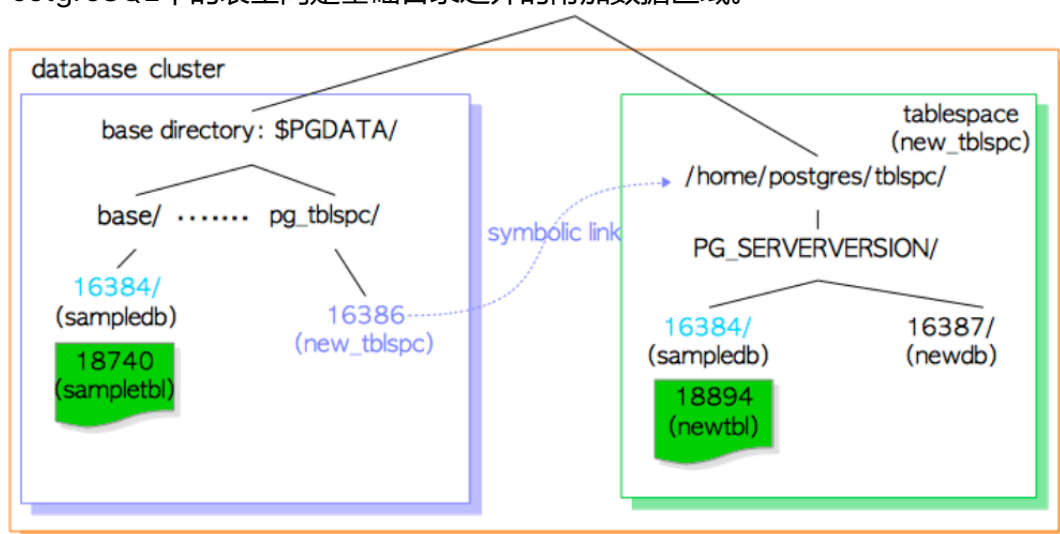
vm -- visual map

在我们update, delete, 行后, 这一行tuple 并不会马上被清理掉, 而是要通过 vacuum , autovacuum 等操作将这些dead tuple 来清理, vm文件的主要作用是显示占用的tuple , 扫描的时候会跳过这些tuple。

对于系统的vacuum 有两种方式 1 Lazy VACUUM 2 Full Vacuum 两种方式对比的不同, 在于第一种不会将系统空间释放给操作系统, 并且对系统的影响会较小, 而第二种方式会将释放的空间给操作系统, 并且早操作的过程中, 会产生一个独占锁, 而影响正在运行的系统。

表空间的布局

PostgreSQL中的表空间是基础目录之外的附加数据区域。



在/home/postgres/tblspc中创建一个表空间newtblspc, 其oid为16458, 则会在表空间下创建一个名如“PG主版本号_目录版本号”的子目录。

```
CREATE TABLESPACE new_tblspc LOCATION '/home/postgres';

select *, oid from pg_tablespace where spcname = 'new_tblspc';
  spcname | spcowner | spcacl | spcoptions | oid
-----+-----+-----+-----+-----
new_tblspc |      10 |      |      | 16396
(1 row)
```

```
ll /home/postgres
total 0
drwx----- 2 postgres postgres 6 Mar 13 23:20 PG_10_201707211
```

表空间目录通过pg_tblspc子目录中的软链接寻址，链接名称与表空间oid相同。

```
#data 目录
ll pg_tblspc/
total 0
lrwxrwxrwx 1 postgres postgres 22 Mar 13 20:59 16385 -> /var/lib/pgsql/10/data
lrwxrwxrwx 1 postgres postgres 14 Mar 13 23:20 16396 -> /home/postgres
```

```
create database tt03 tablespace new_tblspc;

select datname, oid from pg_database where datname = 'tt03';
  datname | oid
-----+-----
tt03      | 16397
(1 row)
```

```
ls -l /home/postgres/PG_10_201707211/
total 12
drwx----- 2 postgres postgres 8192 Mar 13 23:25 16397
```

如果在new_tblspc表空间内创建一个新表，但新表所属的数据库却创建在基础目录下（比如postgres数据库），那么PostgreSQL会首先在new_tblspc表空间内版本特定的子目录下创建目录名称与现有数据库oid相同的子目录，然后将新表文件放置在这个目录下。

```
create table tt04(id int) tablespace new_tblspc;
CREATE TABLE

postgres=# select relname, oid from pg_class where relname='tt04';
  relname | oid
-----+-----
tt04      | 16398
(1 row)
```

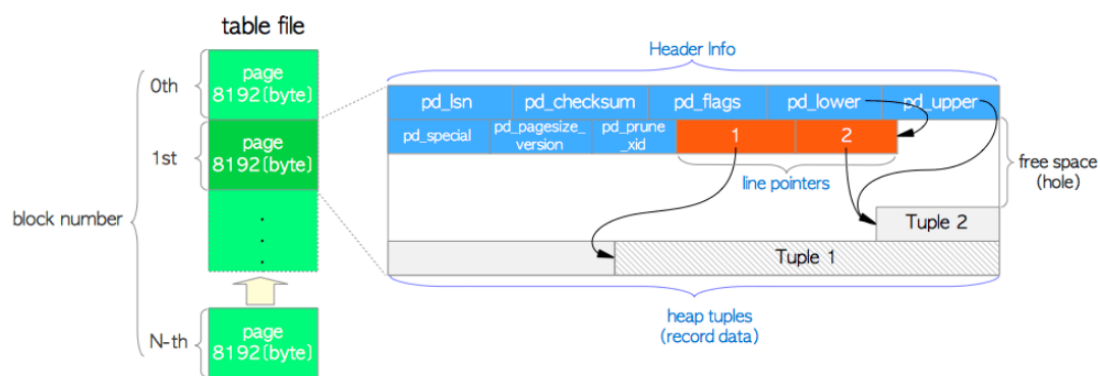
```
select datname, oid from pg_database order by oid;
 datname | oid
-----+-----
 template1 |      1
 template0 |   13213
 postgres  |   13214
 tt03      |   16397
(4 rows)
```

```
ll /home/postgres/PG_10_201707211/
total 12
drwx----- 2 postgres postgres 19 Mar 13 23:30 13214
drwx----- 2 postgres postgres 8192 Mar 13 23:25 16397

ll /home/postgres/PG_10_201707211/13214/
total 0
-rw----- 1 postgres postgres 0 Mar 13 23:30 16398
```

堆表文件的内部布局

数据文件（堆表、索引，也包括空闲空间映射和可见性映射）内部被划分为固定长度的页，或者叫区块，大小默认为8192B（8KB）。每个文件中的页从0开始按顺序编号，这些数字称为区块号。如果文件已填满，PostgreSQL就通过在文件末尾追加一个新的空页来增加文件长度。页面内部的布局取决于数据文件的类型。



表的页面包含了三种类型的数据：

1. 堆元组——即数据记录本身。它们从页面底部开始依序堆叠。
2. 行指针——每个行指针占4B，保存着指向堆元组的指针。它们也被称为项目指针（item pointer）。行指针形成一个简单的数组，扮演了元组索引的角色。每个索引项从1开始依次编号，称为偏移号。当向页面中添加新元组时，一个相应的新行指针也会被放入数组中，并指向新添加的元组。
3. 首部数据——页面的起始位置分配了由结构PageHeaderData定义的首部数据。它的大小为24B，包含关于页面的元数据。

结构体PageHeaderData定义于src/include/storage/bufpage.h。该结构的主要成员变量如下：

- pd_lsn——本页面最近一次变更所写入的xlog记录对应的lsn。它是一个8B无符号整数，与WAL机制有关。
- pd_checksum——本页面的校验和值。
- pd_lower、pd_upper——pd_lower指向行指针的末尾，pd_upper指向最新堆元组的起始位置。
- pd_special——在索引页中会用到该字段，在堆表页中它指向页尾。（在索引页中它指向特殊空间的起始位置，特殊空间是仅由索引使用的特殊数据区域，包含特定的数据，具体内容依索引的类型而定）

行指针的末尾与最新元组起始位置之间的空余空间称为空闲空间或空洞。为了识别表中的元组，数据库内部会使用元组标识符（tuple identifier, TID）。TID由一对值组成，分别是元组所属页面的区块号和指向元组的行指针的偏移号。

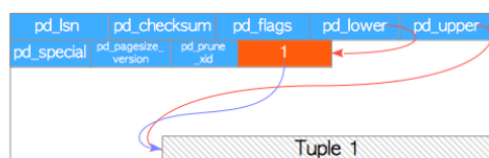
此外，大小超过约2KB（8KB的四分之一）的堆元组会使用一种称为TOAST（The Oversized-Attribute Storage Technique，超大属性存储技术）的方法来存储与管理。

读写元组的方式

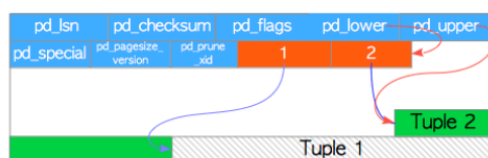
写入元组

假设有一个表仅由一个页面组成，且该页面只包含一个堆元组。此页面的pd_lower指向第一个行指针，而该行指针和pd_upper都指向第一个堆元组。

(a) Before insertion of Tuple 2



(b) After insertion of Tuple 2



当写入第二个元组时，它会被放在第一个元组之后。第二个行指针写入到第一个行指针的后面，并指向第二个元组。pd_lower更改为指向第二个行指针，pd_upper更改为指向第二个堆元组。

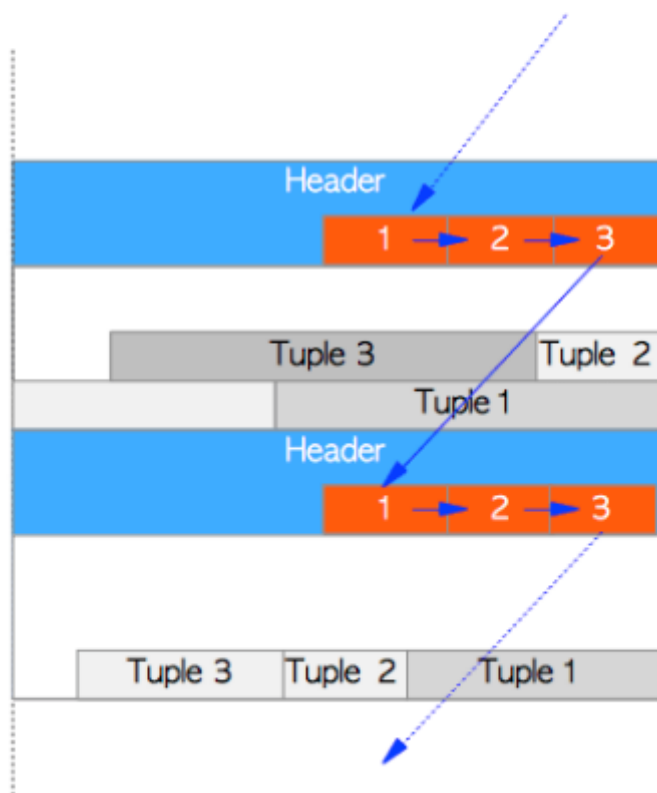
页面内的首部数据也会被改写为适当的值。

读取堆元组

- 顺序扫描——通过扫描每一页中的行指针，依序读取所有页面中的所有元组。

(a) Sequential Scan

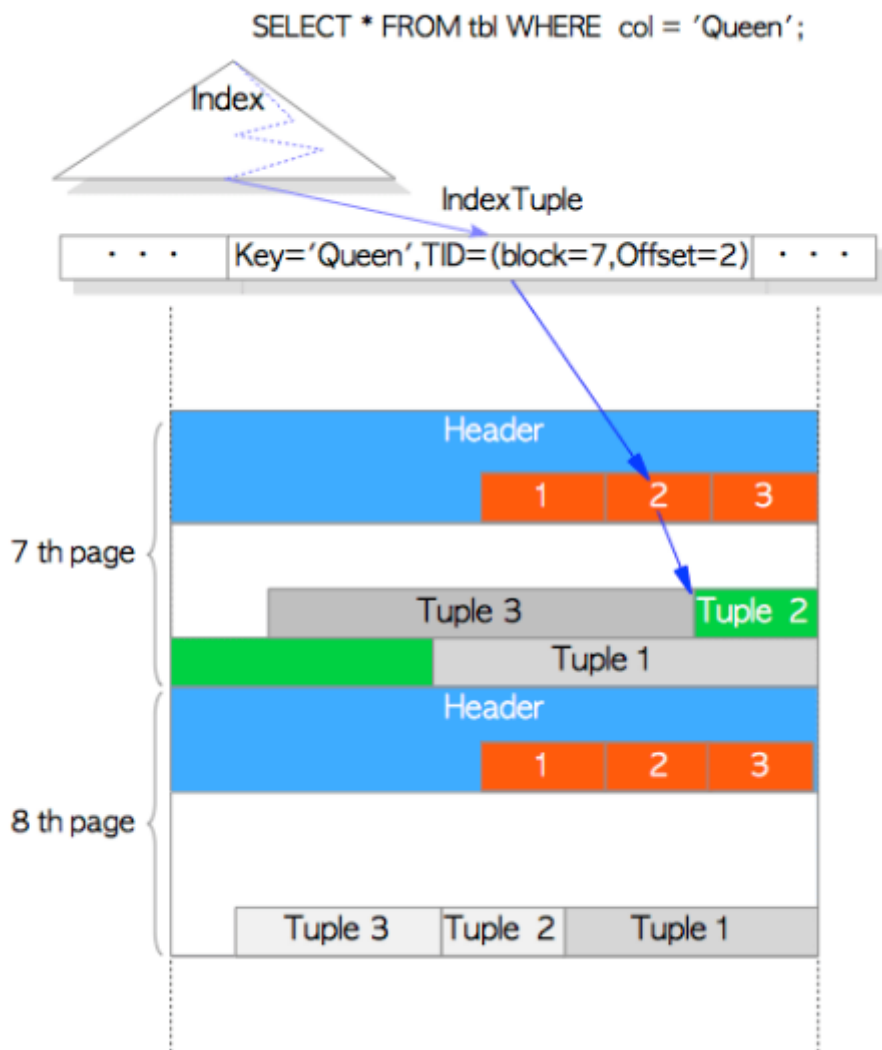
SELECT * FROM tbl;



- B树索引扫描——索引文件包含索引元组，索引元组由一个键值对组成，键为被索引的列值，值为目标堆元组的TID。进行索引查询时，首先使用键进行查找，如果找到了对应的索引元

组，PostgreSQL会根据相应值中的TID来读取对应的堆元组。

(b) Index Scan



```
select ctid,id from tt01 where ctid='(1222,2)';
```

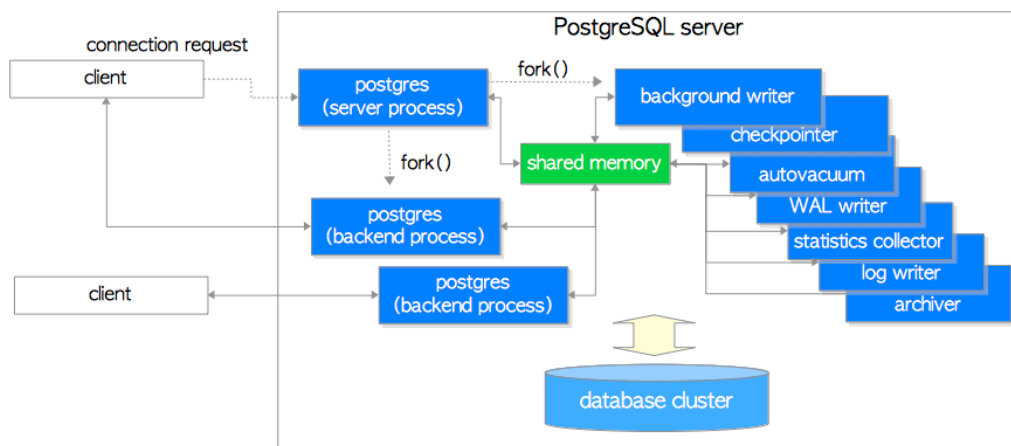
ctid	id
(1222,2)	276174

(1 row)

进程架构

启动PG数据库时，会先启动一个postmaster的主进程，并且会fork出一些辅助的子进程，Postmaster是整个数据库实例的总控进程，负责启动和关闭数据库实例。postmaster命令是一个指向postgres的链接，所以主进程postmaster实际上是第

一个postgres进程。



<https://blog.csdn.net/dazuiba008>

`ps -ux`

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
postgres	1654	0.0	0.0	115548	1208	pts/0	S	Mar13	0:00	-bash
postgres	1718	0.0	0.3	271884	11844	pts/0	S	Mar13	0:00	/usr/pgsql-10/bin/postgres -D /var/lib/pgsql/10/data
postgres	1721	0.0	3.5	272192	138676	?	Ss	Mar13	0:07	postgres: checkpointer process
postgres	1722	0.0	3.4	271884	132788	?	Ss	Mar13	0:01	postgres: writer process
postgres	1723	0.0	0.1	271884	5024	?	Ss	Mar13	0:06	postgres: wal writer process
postgres	1724	0.0	0.0	272300	1788	?	Ss	Mar13	0:00	postgres: autovacuum launcher process
postgres	1725	0.0	0.0	126956	860	?	Ss	Mar13	0:00	postgres: stats collector process
postgres	1726	0.0	0.0	272176	1364	?	Ss	Mar13	0:00	postgres: bgworker: logical replication launcher
postgres	2060	0.0	0.0	115548	1536	pts/1	S	Mar13	0:00	-bash
postgres	16248	0.0	0.0	125012	1780	pts/0	S+	Mar13	0:00	psql
postgres	16249	0.0	0.4	272964	18912	?	Ss	Mar13	0:00	postgres: postgres
postgres	[local]					idle				
postgres	19882	0.0	0.0	155452	1876	pts/1	R+	00:07	0:00	ps -ux

- postgres server process
所有进程的父进程，早期的版本叫postmaster。
pg_ctl start执行后，这个进程就会启动，然后，从物理内存中分配内存给shared memory，然后产生很多其他的background processes，等待客户端来连接，每产生一个连接就会生成一个backend process，一个postgres server process只能监听一个端口，默认端口是5432。尽管一台机器可以运行多个server,但是端口必须不同。
- backend processes
通过TCP协议和客户端建立通讯，当客户端断开时，连接消失。允许多个客户端同时连接，连接数由max_connections参数控制
- background worker process

Table 2.1: background processes.

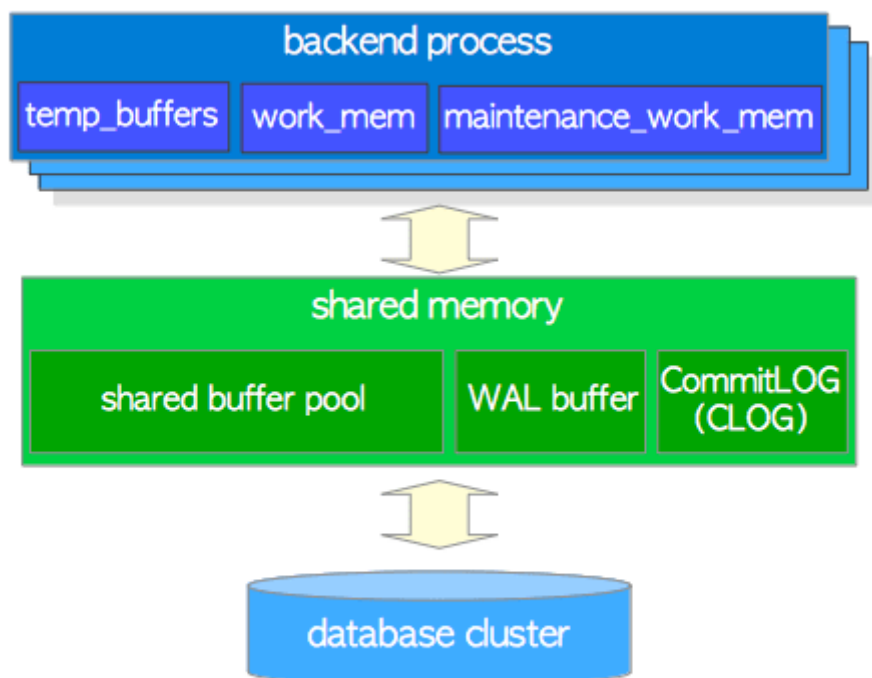
process	description	reference
background writer	进程将shared buffer pool中的脏数据写到磁盘，检查点总能触发这个进程	Section 8.6
checkpointer	在9.2版本以后，检查点会触发产生这个进程	Section 8.6 , Section 9.7
autovacuum launcher	为vacuum process周期性的调用autovacuum work processes	Section 6.5
WAL writer	周期性的从wal buffer刷新数据到磁盘	Section 9.9
statistics collector	收集统计信息进程，比如pg_stat_activity 和 pg_stat_database的数据	
logging collector (logger)	将错误信息写入到日志	
archiver	将日志归档的进程.	Section 9.10

内存架构

在Postgresql中，内存大概被分为两块

Local memory area – 为每一个backend process 分配的内存

Shared memory area – PostgreSQL server 所有的background process使用的内



<https://blog.csdn.net/dazuiba008>

存

- Local memory area

每一个backend process 都会分配一块local memory area, 每一块区域又分为三个子区域, 见下表

Table 2.2: Local memory area

sub-area	description	reference
work_mem	用户在sort,distinct,merge join, hash join的时候会用到这块区域	Chapter 3
maintenance_work_mem	vacuum, reindex会用到这块区域	Section 6.1
temp_buffers	存储临时表会用到这块区域	

- Shared memory area

这块区域在服务器启动的时候分配, 这块区域也是分为好几个子区域, 见下面介绍

Table 2.3: Shared memory area

sub-area	description	reference
shared buffer pool	将表或者索引的page从磁盘加载到shared buffer, 然后在shared buffer操作	Chapter 8
WAL buffer	在服务端出现问题的时候, 确保数据不会丢失, 在写到磁盘之前, wal buffer是wal log的缓存区域	Chapter 9
commit log	为了并发控制所有事物的状态的保持而分配的区域	

另外, Postgresql还分配一些其他的内存区域:

- 为访问控制分配的子区域, 比如轻量级锁, 共享或者专有锁.
- 为其他background process提供的子区域, 比如检查点, vacuum.
- 为事物处理提供的子区域, 比如事物中的保存点, 和二阶段事物提交.